VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Deep Learning and Its Applications (CO3133)

Assignment 2 Report

Deep Learning on Large-Scale Data and Specialized Tasks

Advisor(s): Dr. Lê Thành Sách

Student(s): Cao Đức Thành 2453164

Đặng Thành Phú 2452967

HO CHI MINH CITY, NOVEMBER 2026

Contents

1 Problem and Data Description **5**
1.1 Problem Formulation . . . 5
1.2 Dataset Description . . . 5
1.3 Exploratory Data Analysis . . . 6
1.4 Data Splitting . . . 7
1.5 Preprocessing and Augmentation . . . 7

2 Methodology **8**
2.1 Pipeline Overview . . . 8
2.1.1 Dataset Partitioning and Preprocessing . . . 8
2.1.2 Data Loading . . . 8
2.1.3 Training and Optimization . . . 8
2.1.4 Prediction, Model Selection, and Evaluation . . . 9
2.2 U-Net . . . 10
2.3 Additional experimental loss function . . . 11
2.4 DeepLabV3-ResNet50 (pretrained) . . . 11
2.5 Fine-tuning Strategy . . . 12
2.6 Training Setup . . . 13
2.6.1 Training Configuration and hyperparameter Selection . . . 13
2.6.2 Checkpoint Selection . . . 14
2.7 Experimental Design . . . 14

3 Implementation Results **15**
3.1 Training Behavior . . . 15
3.1.1 U-Net . . . 16
3.1.2 DeepLabV3-ResNet50 with fine-tuning variants . . . 16
3.1.3 Implementation issues and fixes. . . . 17
3.2 Quantitative Results . . . 17
3.2.1 Training and Inference Time . . . 19
3.3 Qualitative Results . . . 19
3.4 Comparison and Discussion . . . 20
3.5 Error Analysis . . . 21
3.6 Limitations and Conclusion . . . 23
3.6.1 Dataset Limitations . . . 23

3.6.2 Preprocessing Trade-off: Evaluation Consistency vs. Distribution Adaptation 24
3.6.3 Architectural and Computational Limitations 24
3.6.4 Experimental and Reproducibility Limitations 25
3.6.5 Conclusion 25

4 AI Usage Disclosure **25**

References **28**



1 Problem and Data Description

1.1 Problem Formulation

A self-driving car needs to know what is in front of it: road, sidewalk, cars, people,... Semantic segmentation solves this by giving a class to every pixel of the camera image.

- Input: one RGB image with shape (3, 512, 1024).
- Output: a label map from 0 to 18 for every pixel, so one class for each pixel
- Prediction unit: the pixel. Pixels with label 255 (void) have no class, so we skip them in the loss and in the metrics

The model give a score for each class at each pixel. We pick the class with the highest score. The task is hard for four reasons:

- Some classes have many more pixels than others
- Thin objects like poles, traffic lights and traffic signs are very small in the image
- Some neightbor classes look alike, for example road and sidewalk, or building, wall and fence
- The test images come from other cities than the training images

1.2 Dataset Description

- Source and license: Cityscapes (https://www.cityscapes-dataset.com), packages gtFine_trainvaltest.zip (252.6 MB) and leftImg8bit_trainvaltest.zip (11.59 GB). Registration is needed; non-commercial use only.
- Content: car videos from 50 cities (mostly Germany). 5,000 frames have fine polygon labels: 2,975 train, 500 val, 1,525 test (test labels are not public). The 34 raw IDs are mapped to 19 classes (road, sidewalk, building, wall, fence, pole, traffic light, traffic sign, vegetation, terrain, sky, person, rider, car, truck, bus, train, motorcycle, bicycle). Everything else, such as the car hood, becomes void (255).
- Format: 2048x1024 RGB PNG images, PNG label maps and JSON polygons. prepare_data.py converts the IDs with a look-up table, and pre_resize.py resizes everything once to 1024x512 (bilinear for images, nearest for masks so no new class IDs appear).

- Bias: one camera, daytime, good weather, European cities. Rare classes have few regions (e.g. only 334 train regions), so their IoU is noisy.

1.3 Exploratory Data Analysis

Numbers come from scripts/m3_eda_stats.py over all images of each split.

1. Class imbalance. Road, building, and vegetation cover 75.69% of training pixels. The 10 rarest classes (motorcycle, rider, train, traffic light, bus, truck, bicycle, wall, traffic sign, fence) cover only 3.52%. Road has 381 times more pixels than motorcycle. The test split is similar.

2. Region size. At 1024x512, over 90% of pole, traffic-light and traffic-sign regions are smaller than 32x32 pixels (92.96%, 94.29%, 90.11%; median 87, 113, 137 px). A car region has a median of 446 px.

3. Void pixels. 11.96% of training pixels are void (11.96 ± 6.77% per image), 9.08% in validation, 12.55% in test. The car hood is always void.

4. Scene complexity. One image has 11.72 classes on average (min 5, max 18).

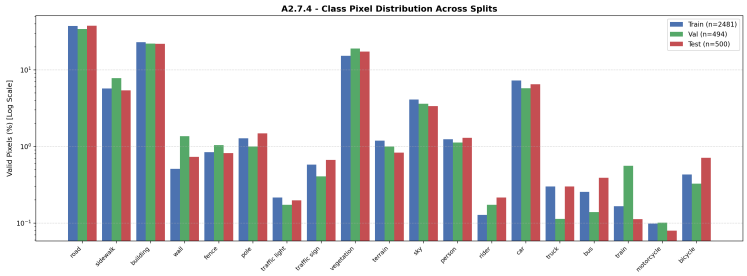
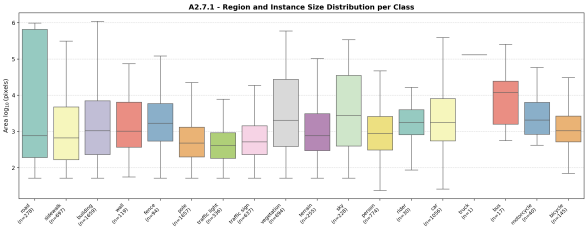
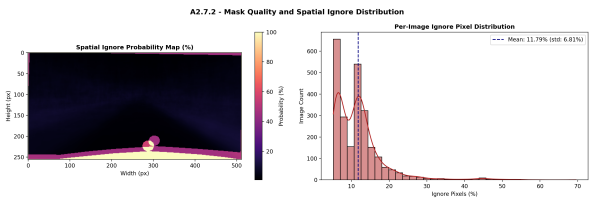


Figure 1.1: Pixel share per class in train, validation and test.



(a) Region size per class ($\log_{10}$ px; first 500 training images).



(b) Where void pixels are, and void share per image.

Figure 1.2: Region size and mask quality.

1.4 Data Splitting

Official test labels are not public, so the official val set is our test set. We take our validation set from the official train set by whole cities (create_splits.py, no randomness). Images of one city come from the same drives and look alike, so a city split stops near-copies from leaking into validation or test.

- Train: 2,481 images, 14 cities (Aachen, Bochum, Bremen, Cologne, Darmstadt, Düsseldorf, Hamburg, Hanover, Krefeld, Mönchengladbach, Strasbourg, Stuttgart, Ulm, Weimar).
- Validation: 494 images, 4 cities (Erfurt, Jena, Tübingen, Zurich). Used for all choices.
- Test: 500 images, 3 cities (Frankfurt, Lindau, Münster). Used once, at the end.

1.5 Preprocessing and Augmentation

Training (src/transforms.py): random 512x512 crop and left-right flip (p=0.5) on image and mask together, colour jitter (0.2) on the image only, then ImageNet normalization. Validation and test use the full 1024x512 image with no augmentation. A training batch is float32 [8, 3, 512, 512] with masks int64 [8, 512, 512] (values 0–18 and 255).

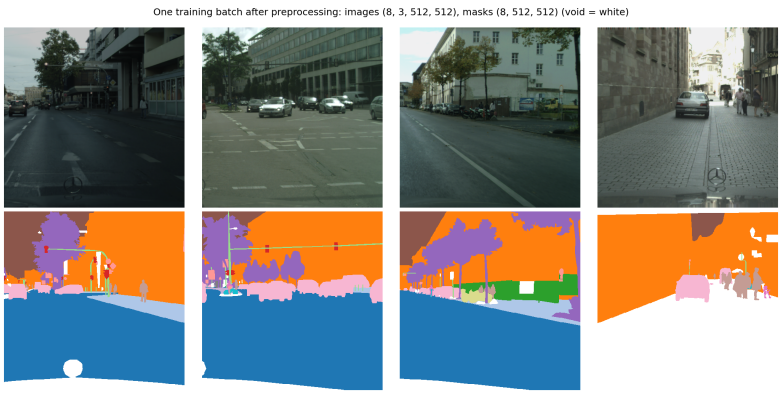


Figure 1.3: One real training batch after preprocessing (top: images, bottom: masks, void in white).

2 Methodology

2.1 Pipeline Overview

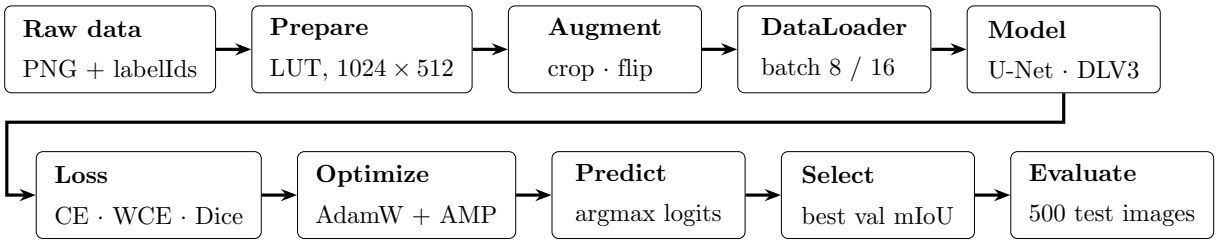


Figure 2.1: One pipeline shared by all six runs.

2.1.1 Dataset Partitioning and Preprocessing

Three scripts run once before training: scripts/prepare_data.py (raw IDs -> 19 trainIds), scripts/pre_resize.py (resize to 1024x512 on disk) and scripts/create_splits.py (city split, written to splits/train.txt, val.txt, test.txt). Resizing once saves time, because the loader does not have to shrink 2048x1024 images in every epoch. The online part (SegTransform: crop, flip, jitter, normalization) runs on every sample.

2.1.2 Data Loading

CityscapesSeg (src/data.py) reads the file names from a split file and returns one (image, mask) pair after SegTransform. PyTorch DataLoader settings:

- Batch size 8 and 4 workers for the local runs (U-Net, WCE, CE + Dice); batch size 16 and 6 workers for M1–M3 on Colab (Table 2.4). Pinned memory, workers kept alive between epochs.
- Train: shuffled with a seeded generator, and every worker gets its own fixed seed, so the batch order can be repeated. The last small batch is dropped (drop_last).
- Validation and test: no shuffle, nothing dropped, so every image is scored.

2.1.3 Training and Optimization

Loss function:

- **Objective:** Pixel-wise cross-entropy over the 19 Cityscapes classes; pixels with the ignore index (void or unlabeled) contribute nothing to the loss or gradients.



- **Class weighting:** None in the CE runs; all classes contribute equally (the weighted variant used in E2 is described in Section 2.3). **Reduction:** for each batch, the loss is the mean over all labelled pixels (pixels with ignore index 255 are excluded). The epoch loss in the logs is the mean of the batch losses, weighted by the number of images in each batch.

Optimizer:

- **AdamW:** All models use AdamW with its default settings except for the learning rates listed in Table 2.4. AdamW adapts each parameter step size and applies default weight decay as a regularizer.
- **Differential learning rates:** For M2 and M3, the pretrained backbone uses a lower learning rate than the new head (Table 2.4), helping preserve ImageNet features while the head adapts to Cityscapes. M1 keeps the backbone frozen, while U-Net does not use the DeepLabV3 backbone–head structure.

Mixed precision:

- **FP32 vs. FP16:** FP32 (32-bit) is the standard number format; FP16 (16-bit) uses half the memory and is faster on GPUs, but can only represent a smaller range of values.
- **AMP:** Used for all six runs (enabled when `amp: true` and a CUDA GPU is available). The forward pass and loss are placed inside `torch.autocast`, which picks the precision for each operation: heavy operations such as convolutions run in FP16, while sensitive ones such as BatchNorm, softmax and the loss stay in FP32. The backward pass follows the same precision automatically.
- **GradScaler:** Scales the loss before backpropagation to reduce FP16 gradient underflow, then unscales the gradients before the optimizer step.

2.1.4 Prediction, Model Selection, and Evaluation

Prediction:

- **Input:** Images are resized to 512×1024 and normalized, with no cropping or augmentation; the full image is predicted in one forward pass.
- **Inference mode:** The model runs in `eval()` mode under `torch.inference_mode()`, so no gradients are computed and BatchNorm uses its stored statistics. AMP is used as in training.

- **Class decision:** The model outputs 19 scores per pixel; the predicted class is the one with the highest score (arg max).

Evaluation procedure:

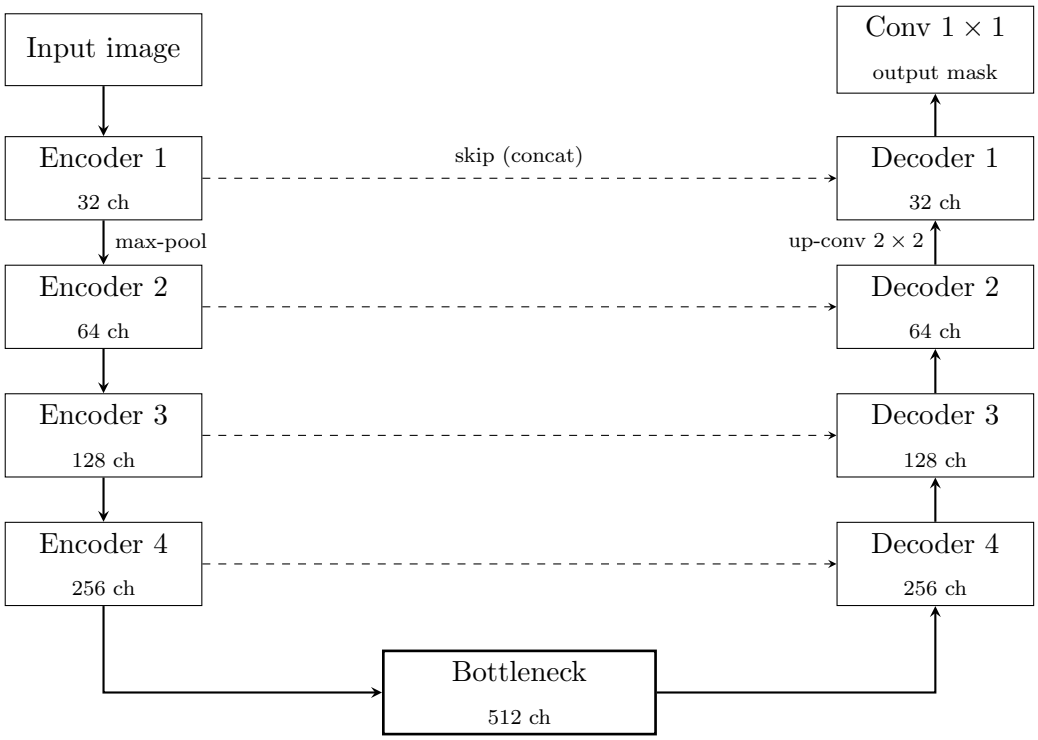
- Validation and test use the same procedure; only the data split differs (Table 2.1).
- **Confusion matrix:** Predictions and ground truth are accumulated over the whole split into a 19×19 confusion matrix; pixels labelled 255 are excluded. From it, true positives (TP), false positives (FP) and false negatives (FN) are obtained for each class.
- **Metrics:**
  - IoU per class $= \frac{TP}{TP+FP+FN}$; $mIoU$ is the mean over classes (primary metric).
  - $Dice$ per class $= \frac{2TP}{2TP+FP+FN}$; *mean Dice* is the mean over classes.
  - $Pixel\ accuracy = \frac{\sum TP}{\text{number of valid pixels}}$.
  - Classes that never appear in either the prediction or the ground truth are excluded from the means.
- **Loss:** The cross-entropy loss on the split is also reported.

Table 2.1: Use of the validation and test splits

	Validation	**Test**
Images	494	500
When	After every epoch	Once, after training
Model evaluated	Current model	`best.pt`
Purpose	Model selection: `best.pt` is saved whenever validation mIoU improves	Final reported results

2.2 U-Net

Baseline, our code, src/models/unet.py. Four encoder stages of two [3×3 conv, BatchNorm, ReLU] blocks with max-pooling (channels 32→256, bottleneck 512), a decoder with 2×2 transposed convolutions and skip connections, and a 1×1 output conv. 7,763,635 parameters. *Good:* keeps fine details, small and fast. *Weak:* no pretrained knowledge, small view of the image.



2.3 Additional experimental loss function

All three losses are our code (src/losses.py) and skip label 255.

1. CE: standard cross-entropy. Used for the U-Net and for E0/E1.
2. Weighted CE: rare classes get a bigger weight $w_c = 1 + \log(\max_k N_k / N_c)$, where N_c is the pixel count of class c (road 1.00, motorcycle 6.94).
3. CE + Dice (λ=1), with $\mathcal{L}_{\text{Dice}} = 1 - \frac{1}{|\mathcal{C}_b|} \sum_{c \in \mathcal{C}_b} \frac{2 \sum p_c g_c + 1}{\sum p_c + \sum g_c + 1}$ over the classes $\mathcal{C}_b$ in the batch. Dice scores the overlap of each class, so small classes count as much as big ones.

2.4 DeepLabV3-ResNet50 (pretrained)

DeepLabV3 is used through the `torchvision` implementation `deeplabv3_resnet50`. It consists of a pretrained ResNet-50 backbone that extracts features and an ASPP head that converts them into per-pixel class scores[1].

Backbone (ResNet-50):

- **Pretraining:** Initialized with weights pretrained on ImageNet (`IMAGENET1K_V1`) and adapted to Cityscapes through fine-tuning.

Table 2.2: Tensor shapes for two DeepLabV3 input resolutions.

Tensor	**Train** (512×512 **input**)	**Evaluation** (512×1024 **input**)
Input shape	`(1, 3, 512, 512)`	`(1, 3, 512, 1024)`
Backbone output shape	`(1, 2048, 64, 64)`	`(1, 2048, 64, 128)`
Model output shape	`(1, 19, 512, 512)`	`(1, 19, 512, 1024)`

  - **Dataset:** ImageNet-1K with 1000 categories[1].
  - **Task:** Image classification, with one label per image at 224×224 resolution; the weights reach 76.1% top-1 accuracy on ImageNet.[1]
  - **Parameters:** The original ResNet-50 has 25.6M parameters[1]. But in this experiment only have 23.5M (verified with torch) pretrained parameters in the backbone .

- **Backbone stages:** ['conv1', 'bn1', 'relu', 'maxpool', 'layer1', 'layer2', 'layer3', 'layer4'] (torch)
- **Feature extractor:** The original ImageNet classification layer is removed; the backbone output is passed to the new ASPP head.

Head (ASPP):

- **Structure:** Atrous Spatial Pyramid Pooling (ASPP) from DeepLabV3. It looks at the backbone features at several scales at once, using parallel convolutions with different dilation rates and one branch that sees the whole image, so the model can capture both small and large objects.
- **Initialization:** Randomly initialized and trained from scratch on Cityscapes; no pretrained DeepLabV3 weights are used.
- **Parameters:** 16.1M parameters (torch).

Size: 39.6M parameters in total: 23.5M in the backbone and 16.1M in the head.

2.5 Fine-tuning Strategy

To study how much of the pretrained backbone should be adapted to Cityscapes, three variants are trained that differ only in which layers are updated (Table 2.3). All other settings are shared (Table 2.4).

- **M1 – Frozen backbone:** Only the head is trained, and the ImageNet features are used as they are. The frozen backbone is kept in `eval()` mode so its BatchNorm

Table 2.3: DeepLabV3-ResNet50 fine-tuning variants

	M1 (Frozen)	**M2 (Partial)**	**M3 (Full)**
`conv1`, `bn1`, `relu`, `maxpool`	Frozen	Frozen	Trained
`layer1`–`layer2`	Frozen	Frozen	Trained
`layer3`–`layer4`	Frozen	Trained	Trained
ASPP head	Trained	Trained	Trained
Trainable parameters	16.1M	38.2M	39.6M

statistics are not changed. This is the cheapest option and serves as a baseline for how well ImageNet features transfer without adaptation.

- **M2 – Partial fine-tuning:** conv1, bn1, relu, maxpool, `layer1`–`layer2` stay frozen. `layer3`–`layer4`, which learn task-specific high-level features, are fine-tuned together with the head.
- **M3 – Full fine-tuning:** nothing is frozen: the stem (conv1, bn1), all four backbone stages and the head are trained. This gives the model the most freedom to adapt to urban street scenes, at the highest risk of overwriting useful pretrained features.
- **Differential learning rates:** In M2 and M3, the backbone is updated with a smaller learning rate than the head (Table 2.4). The randomly initialized head needs large updates, while the pretrained backbone only needs small adjustments. M3 uses an even smaller backbone rate than M2 because it also updates the early layers.
- **Implementation:** Frozen layers have `requires_grad=False` and are excluded from the optimizer.

2.6 Training Setup

2.6.1 Training Configuration and hyperparameter Selection

Hardware configuration:

- U-Net and the two E2 loss variants (WCE, CE + Dice) were trained locally on an RTX 5070 Ti (16 GB) with batch size 8. The three DeepLabV3 variants M1–M3 were trained on Google Colab with an RTX PRO 6000 (95 GB); the larger GPU memory allowed batch size 16 (Table 2.4).

Table 2.4: Models trained and their hyperparameters

Hyperparameter	**U-Net**	**DeepLabV3-M1 (Frozen)**	**DeepLabV3-M2 (Partial)**	**DeepLabV3-M3 (Full)**
GPU	RTX 5070 Ti	RTX PRO 6000	RTX PRO 6000	RTX PRO 6000
Epochs	40	60	60	60
Batch size	8	16	16	16
Optimizer	AdamW			
Learning rate (head / whole model)	10^{-3}	10^{-4}	10^{-4}	10^{-4}
Backbone learning rate	–	frozen	10^{-5}	10^{-6}
Weight decay	0.01			
LR scheduler	None (constant)			
Loss	Cross-entropy			
Ignore index	255			
Mixed precision (AMP)	Yes			
Random seed	666			
Input (train)	512×512 random crop from 512×1024			
Steps per epoch	310	155	155	155

2.6.2 Checkpoint Selection

Validation after every epoch. The best.pt saved on the best validation mIoU which was used to test.

2.7 Experimental Design

Three experiments are conducted (Table 2.5). Each changes one main factor, and each has a decision rule fixed in advance, evaluated on the validation set.

- **Weighted CE:** Each class is weighted by $w_c = 1 + \log(N_{\max}/N_c)$, where N_c is the number of training pixels of class c, so rare classes contribute more to the loss.
- **CE + Dice:** Unweighted CE plus Dice loss with $\lambda = 1$; the Dice term directly optimizes per-class overlap.

Table 2.5: Experimental design

Experiment	**Changed factor**	**Held fixed**	**Not matched**	**Hypothesis → decision rule**
E0 – Baseline vs. pretrained	U-Net from scratch vs. DeepLabV3-M1 (frozen) with ImageNet backbone	Data split, preprocessing (512×1024 resize, 512×512 crops), AdamW, CE loss, seed	Epochs (40 / 60), learning rate, batch size (8 / 16), GPU	DeepLabV3-M1 (frozen) beats U-Net by ≥ 5 mIoU points val mIoU
E1 – Fine-tuning depth	Trainable backbone: frozen (M1) / partial (M2) / full (M3)	DeepLabV3, CE loss, head LR 10^{-4}, 60 epochs, batch 16, GPU, seed, data	Backbone LR (partial 10^{-5}, full 10^{-6})	Choose m2-partial or m3-full if it is ≥ 2 mIoU points val mIoU above frozen
E2 – Loss variant	Loss on DeepLabV3-M2 (partial fine-tuning): CE vs. weighted CE vs. CE + Dice	DeepLabV3-M2 (partial: layer3 + layer4 + head), head LR 10^{-4}, backbone LR 10^{-5}, 60 epochs, seed, data	Batch size (CE: 16, others: 8), GPU (CE: RTX PRO 6000, others: RTX 5070 Ti)	Weighted CE or CE + Dice adds ≥ 1 mIoU points val mIoU over CE

- **Rare-10 IoU:** Mean IoU over the 10 classes with the fewest training pixels: motorcycle, rider, train, traffic light, bus, truck, bicycle, wall, traffic sign and fence.
- **Limitations:** E0 and E2 are not fully controlled (see the *Not matched* column); in E2, part of any difference may come from the batch size rather than the loss. Each configuration was trained once with a single seed, so run-to-run variance is not measured.

3 Implementation Results

3.1 Training Behavior

Curves come from each run's log.csv (summary by scripts/m3_training_summary.py). With no LR decay, validation mIoU still moves by up to 2.6 pp between epochs in the last 20 epochs for DeepLabV3 (U-Net up to 4.1 pp).

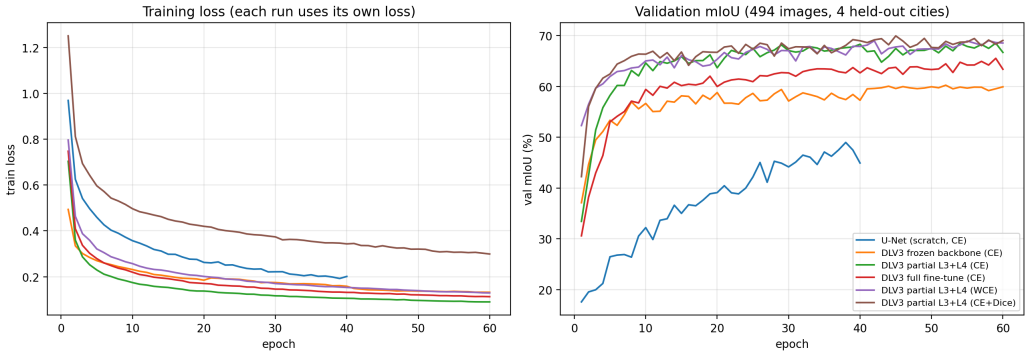


Figure 3.1: Training loss (left; each run has its own loss) and validation mIoU (right).

3.1.1 U-Net

- Slow learning: validation mIoU is 17.6% after epoch 1, 32.2% after epoch 10 and 39.1% after epoch 20. Best 48.97% at epoch 38 (`best.pt`); the last epoch drops to 44.92%, which shows the noise.
- Underfitting: the model never passes 50% mIoU, and the best epoch (38 of 40) is close to the end, so the curve has not plateaued. Training loss goes from 0.969 to 0.201.
- Some over-confidence: validation loss is lowest at epoch 26 (0.298) and then rises to 0.348, while mIoU still improves. Selecting by mIoU avoids this problem.
- Time: 1,225 s for 40 epochs (median 30.3 s per epoch, RTX 5070 Ti).

3.1.2 DeepLabV3-ResNet50 with fine-tuning variants

All three variants use the hyperparameters listed in Table 2.4 and differ only in how much of the ImageNet-pretrained ResNet-50 backbone is trained (M1 frozen, M2 partial, M3 full) and, accordingly, in the backbone learning rate.

Learning curves. Because all variants start from an ImageNet-pretrained backbone, they improve significantly in the first 10 epochs: validation mIoU rises from 37.1% to 56.7% for M1 (+19.6), from 33.4% to 64.6% for M2 (+31.2) and from 30.6% to 59.4% for M3 (+28.8), while the training loss drops sharply. After epoch 10 progress becomes slow and the curves fluctuate: over the remaining 50 epochs, validation mIoU gains only +3.6 (M1), +3.9 (M2) and +6.1 (M3) points more. M2 stays ahead of the other variants throughout this phase.

Selected checkpoint. For each variant we keep the epoch with the highest validation mIoU: epoch 52 for M1 (60.25%), epoch 59 for M2 (68.54%) and epoch 59 for M3 (65.54%). We select on mIoU rather than validation loss because the two disagree: validation loss reaches its minimum much earlier (epochs 25, 30 and 52) while mIoU keeps improving. A likely explanation is that the model keeps correcting boundary and small-object pixels, which raises mIoU, while becoming more confident on the pixels it still gets wrong, which raises the loss.



Overfitting and underfitting. M1 overfits mildly: after epoch 25 its validation loss rises from 0.268 to 0.295 while its training loss keeps falling, giving the widest train–validation gap (0.163). M2 and M3 keep a smaller gap (about 0.11) and do not overfit. M3 instead underfits: its best epoch is 59 of 60 and its validation mIoU has not plateaued, likely because its backbone learning rate (10^{-6}) is too small. M2 also peaks at epoch 59, so both fine-tuned variants might benefit from more epochs.

3.1.3 Implementation issues and fixes.

- **Frozen BatchNorm statistics.** When the backbone is frozen (M1), its BatchNorm layers are switched to evaluation mode during training so that the running statistics learned on ImageNet are not overwritten by small Cityscapes batches.
- **Differential learning rates.** The head and backbone are placed in separate optimiser parameter groups, so the randomly initialised head can train at 10^{-4} while the pretrained backbone is updated more gently (10^{-5} or 10^{-6}).
- **Single-sample batches.** The ASPP image-pooling branch applies BatchNorm to a 1×1 feature map, which fails for a batch of one image. With 2,481 training images and batch size 16 (or 8), the last batch holds a single image, so batches of size 1 are skipped (2,480 images are used per epoch).
- **Label-preserving resize.** Masks are resized with nearest-neighbour interpolation to avoid creating invalid class IDs at object boundaries.

3.2 Quantitative Results

All numbers in this part come from files in the assignment2/ folder: log.csv and metrics_test.json of each run in checkpoints/, and the outputs of scripts/m3_training_summary.py, benchmark_cost.py and m3_error_qual.py in docs/figures/m3/.

Table 3.1: All runs. Validation = best epoch (494 images); test = 500 images, tested once. Loss depends on each run's loss function, so it is not comparable across losses.

		Validation				Test			
Run (model,loss)	Ep.	mIoU	Dice	PixAcc	Loss	mIoU	Dice	PixAcc	Loss
U-Net, CE	38	48.97	60.66	90.81	0.304	49.35	60.86	91.65	0.278
DLV3 frozen, CE	52	60.25	73.39	91.80	0.284	57.01	70.21	91.55	0.287
DLV3 full, CE	59	65.54	77.79	93.24	0.222	61.10	73.83	93.05	0.227
DLV3 partial, CE	59	68.54	80.17	94.29	0.197	66.53	78.48	94.07	0.201
DLV3 partial, WCE	58	69.02	80.63	93.99	0.339	67.21	79.10	93.90	0.344
DLV3 partial, CE+Dice	45	69.96	81.31	94.17	0.481	68.09	79.81	93.92	0.537

Table 3.2: Test IoU (%) per class. * = one of the 10 rarest classes. (rarity base on train data, counting pixels)

Class	U-Net	Frz	Full	Part	WCE	C+D	Class	U-Net	Frz	Full	Part	WCE	C+D
road	95.23	95.27	96.33	97.13	96.83	96.92	sky	89.51	88.44	91.52	92.77	91.95	92.11
sidewalk	68.02	66.40	72.39	77.62	76.66	76.44	person	59.67	61.25	67.58	71.84	71.60	72.10
building	85.45	85.31	87.78	89.19	89.03	88.91	rider*	22.59	37.50	45.20	51.04	51.38	52.99
wall*	16.38	22.28	27.66	37.49	41.77	38.24	car	85.14	87.77	90.03	91.74	91.48	91.12
fence*	29.98	40.69	42.61	44.88	48.52	48.93	truck*	16.58	56.38	50.82	61.70	65.65	62.51
pole	46.07	33.44	41.99	47.19	48.28	47.55	bus*	31.10	56.13	61.39	72.47	70.97	69.55
traffic light*	33.40	38.31	47.27	52.98	53.02	55.46	train*	3.83	30.78	35.14	48.25	43.66	68.21
traffic sign*	47.85	51.32	58.03	64.42	65.74	65.65	motorcycle*	10.03	39.69	41.59	50.79	54.49	52.48
vegetation	89.13	85.40	88.16	89.54	89.51	89.20	bicycle*	58.37	58.60	63.08	66.78	67.83	69.28
terrain	49.31	48.17	52.33	56.33	58.64	55.98	mIoU	49.35	57.01	61.10	66.53	67.21	68.09

E0 – Baseline vs. pretrained. Rule met. DeepLabV3-M1 (frozen) reaches 60.25% val mIoU vs. 48.97% for the U-Net (+11.28 points, threshold 5). The gain holds on test (57.01% vs. 49.35%, +7.66) and in Dice (+9.35 on test), while pixel accuracy is almost unchanged (91.55% vs. 91.65%), since large classes are already easy for the U-Net.

E1 – Fine-tuning depth. Rule met; partial is chosen. Val mIoU is 60.25% (frozen), 65.54% (full) and 68.54% (partial), so partial is +8.29 points above frozen (threshold 2). Test keeps the same order (57.01%, 61.10%, 66.53%). Full also uses a 10 times smaller backbone LR, so its lower score cannot be attributed to depth alone.

E2 – Loss variant. Rule met by CE + Dice. Over CE (68.54% val mIoU), weighted CE adds +0.48 points (69.02%), below the 1-point threshold, while CE + Dice adds +1.42 points (69.96%), above it. Val Dice follows the same order (80.17%, 80.63%, 81.31%). Batch size and GPU differ from the CE run, and each setting was run once, so these small gains may not be meaningful. Test mIoU keeps the same order (66.53% CE, 67.21% WCE, 68.09% CE + Dice).

Final model: partial + CE + Dice (best validation mIoU, epoch 45).

3.2.1 Training and Inference Time

Table 3.3: Training time from each run's `log.csv`. Train / epoch is the median of `train_seconds`; total train is the sum of `train_seconds`; total val is the sum of `eval_seconds` (494 validation images per epoch).

Model	GPU	Epochs	Train / epoch	Total train	Total val	Total
U-Net	RTX 5070 Ti	40	30.3 s	1,225 s	202 s	1,426 s
DLV3 frozen	RTX PRO 6000	60	14.4 s	873 s	200 s	1,073 s
DLV3 partial	RTX PRO 6000	60	22.6 s	1,356 s	196 s	1,552 s
DLV3 full	RTX PRO 6000	60	25.7 s	1,542 s	196 s	1,738 s
DLV3 partial + WCE	RTX 5070 Ti	60	51.7 s	3,123 s	453 s	3,576 s
DLV3 partial + CE+Dice	RTX 5070 Ti	60	54.9 s	3,311 s	491 s	3,802 s

Inference speed is measured separately with scripts/benchmark_cost.py (cost.json; RTX 5070 Ti, batch 1, 512 × 1024 input, FP16 autocast, median of 200 runs): 6.95 ms per image for the U-Net (about 144 FPS) and 16.5–17.0 ms for the DeepLabV3 variants (about 59–60 FPS).

3.3 Qualitative Results

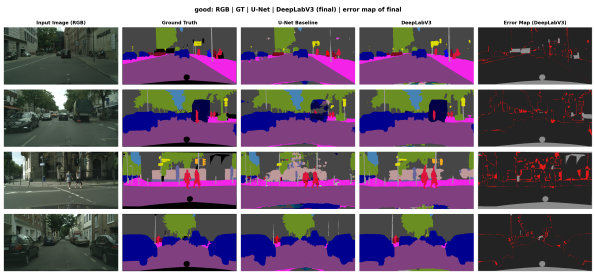
Examples are picked by fixed rules from per-image scores, not by hand. Columns: image | ground truth | U-Net | DeepLabV3 | errors of DeepLabV3 (red).

1. Good (4 best images, Fig. 3.2a): DeepLabV3 0.78–0.83 mIoU vs. U-Net 0.41–0.49. Errors are only thin lines at borders. The U-Net breaks the truck in munster_000012 into pieces.

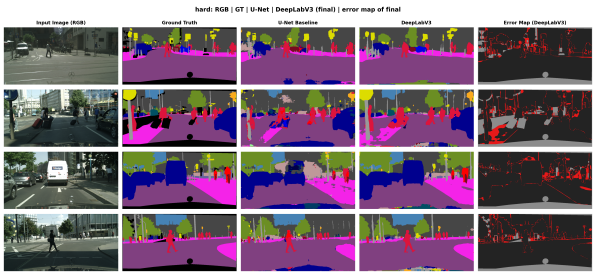
2. Hard but correct (many small objects, biggest gain over U-Net, Fig. 3.2b): 117–150 small regions per image; DeepLabV3 0.62–0.71 vs. U-Net 0.33–0.42. Poles, signs and people are found.

3. Failures (4 lowest pixel accuracy, 61.3–78.3%, Fig. 3.2c): big stone walls become building/vegetation (Lindau), a wide sidewalk becomes road (frankfurt_000001_017459), a construction fence becomes wall/building (frankfurt_000001_012738).

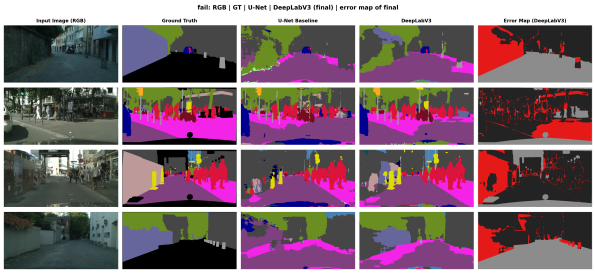
4. No-label test images (Fig. 3.2d): we can only show where the models disagree. DeepLabV3 looks cleaner but still makes clear mistakes (part of a truck labelled person).



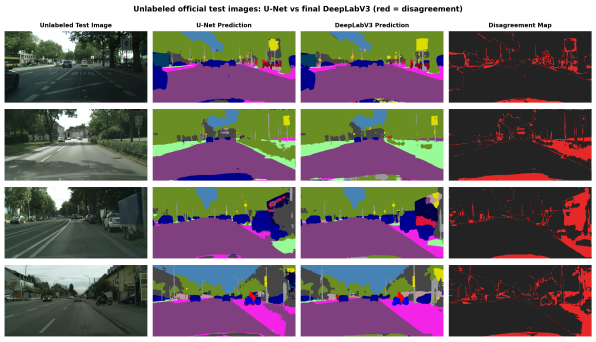
(a) Good.



(b) Hard but correct.



(c) Failures.



(d) No labels (red = models disagree).

Figure 3.2: U-Net vs. final DeepLabV3 (partial, CE+Dice).

3.4 Comparison and Discussion

What helps most. On validation mIoU, pretraining gives +11.28 points (E0), partial fine-tuning a further +8.29 (E1), and the loss at most +1.42 (E2). Most of the gain comes from pretraining and fine-tuning, not from the loss.



Final model. CE + Dice gives the highest validation mIoU (69.96%) and passes the E2 rule. It is also the best run on test (68.09% mIoU vs. 66.53% for DeepLabV3-M2 with CE), so it is used as the final model.

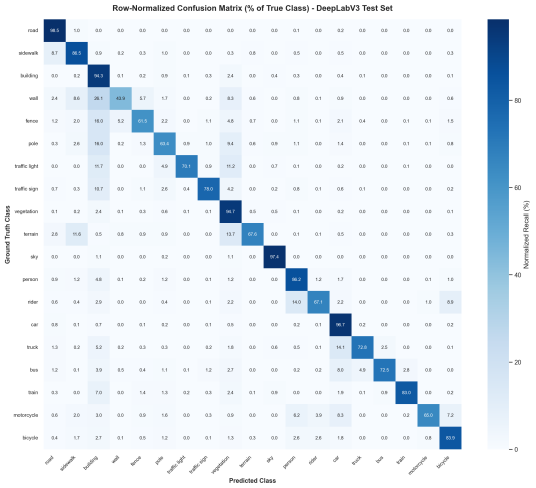
Metrics and splits. Pixel accuracy barely changes across runs (90.81–94.29%) because large classes dominate it, while mIoU ranges from 48.97% to 69.96%. All DeepLabV3 runs score 1.8–4.4 points lower on test than on validation, but the model ranking is the same on both splits.

Cost. DeepLabV3 has about five times more parameters than the U-Net and is about 2.4 times slower at inference (16.5 vs. 6.95 ms per image, Section 3.2.1), but still runs at about 60 images per second and gains +18.74 test mIoU points with the final model (+17.18 with CE).

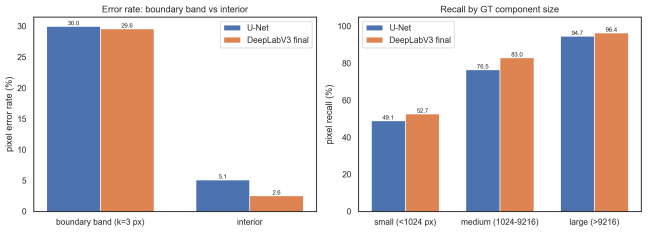
GPU utilization. Although the RTX 5070 Ti and RTX PRO 6000 have large VRAM and many CUDA cores, our runs did not use them at full capacity: batch sizes were small (8 and 16) and only 4 (local) or 6 (Colab) data-loading workers were used, so data loading rather than computation may limit the speed. The reported training times therefore reflect our settings, not the full capability of each GPU.

3.5 Error Analysis

We run scripts/m3_error_qual.py on the final model (it gives the same 68.09% mIoU as evaluate.py). Each of the 13.93M wrong pixels goes into one group, checked in order: (1) *boundary* (within 3 px of a class border), (2) *small region* (region <1,024 px), (3) *related-class confusion* (true and predicted class in the same family, e.g. {road, sidewalk, terrain}, {building, wall, fence}, {car, truck, bus, train}), (4) *other*.



(a) Confusion matrix (% of each true class).



(b) Error at borders vs. inside; recall by region size.

Figure 3.3: Error analysis on the test set (U-Net shown in (b) for comparison).

Table 3.4: Most common mistakes of the final model (test).

GT	Predicted	Pixels	% of GT	% of errors	Worst image
building	vegetation	1,222,575	2.43	8.78	lindau_000014
sidewalk	road	1,077,002	8.70	7.73	frankfurt_000001_017459
vegetation	building	960,800	2.42	6.90	lindau_000048
road	sidewalk	876,898	1.02	6.29	lindau_000024
pole	building	542,103	15.99	3.89	lindau_000050
building	pole	462,311	0.92	3.32	frankfurt_000001_038844
wall	building	437,984	26.10	3.14	lindau_000047

Table 3.5: Error groups of the final model (shares of all wrong pixels).

Group	How often	Examples	Cause → idea
1. Boundary	63.3%, though borders are only 13.0% of pixels. Error 29.6% at borders vs. 2.56% inside (U-Net 30.0% vs. 5.11%).	frankfurt_ 000001_066574, munster_000159	Small output map is enlarged, so borders blur; labels are also rough at edges. → DeepLabV3+ decoder, border loss, full-size training.
2. Small regions	2.4%. Recall 52.7% small vs. 96.4% large regions; pole/light/sign recall 70.5%.	frankfurt_ 000001_028335, _055709	Lost when the network shrinks the image. → bigger input, multi-scale testing.
3. Related classes	19.2%. Only 43.9% of wall is found (26.1% → building); sidewalk → road 8.7%.	frankfurt_ 000001_017459, _012738	Similar look; the class depends on 3D shape. → more context, more wall/fence samples.
4. Other	15.1%, e.g. building ↔ vegetation.	lindau_000047, lindau_000014	Mixed regions (walls under trees); Lindau looks unlike training cities. → more varied data.
Imbalance (per class)	Rare-10 classes: 4.2% of pixels but 22.2% of errors; IoU 58.3% vs. 91.7% for road/building/vegetation.	Table 3.2	Few training examples; WCE/Dice helped only a little. → sample rare classes more, more data, several seeds.

3.6 Limitations and Conclusion

3.6.1 Dataset Limitations

- One camera, daytime, good weather, European cities. We cannot say how the models work at night, in rain or in other countries.
- Only three test cities. Lindau looks unlike the training cities and causes many of the worst images (Table 3.4).
- Rare classes have few regions (e.g. 334 *train* regions), so their IoU changes a lot from run to run.

- Labels are rough at object edges, so part of the boundary error may be label noise, not model error.

3.6.2 Preprocessing Trade-off: Evaluation Consistency vs. Distribution Adaptation

We use one fixed preprocessing for all models: the same 1024×512 files, the same ImageNet mean/std, and the full image with no augmentation at test time. This keeps the comparison fair: a gap in mIoU comes from the model, not from different inputs. It also matches what the pretrained ResNet-50 expects. The cost is that we do not adapt the input to the data or to each model:

- Normalization: ImageNet statistics are not Cityscapes statistics. For the scratch U-Net, Cityscapes mean/std might fit better. We did not test this.
- Crop vs. full image: we train on 512×512 crops but test on the full 1024×512 image. The model sees more context at test time than in training, and the ASPP global-pooling branch sees a different input size.
- Half resolution: all images are shrunk by 2 for speed. This is the same for all models, but it removes detail of thin objects (median pole region only 87 px) and makes our numbers not comparable with the official benchmark.
- Light augmentation and no test-time tricks: colour jitter of 0.2 does not cover night or bad weather, and we skip flip/multi-scale testing. This keeps inference simple and fast, but leaves some accuracy unused.

3.6.3 Architectural and Computational Limitations

- DeepLabV3 predicts at 1/8 of the input size and has no decoder, so borders are blurry; boundary errors are 63.3% of all errors.
- The U-Net has no pretrained knowledge and a small view of the image, so it underfits (below 50% mIoU).
- DeepLabV3 needs 654 GFLOPs per image, 3.4 times more than the U-Net.
- The local 16 GB GPU limits the local runs (U-Net, WCE, CE + Dice) to batch 8, and all runs use half-size images. We did not try bigger backbones or DeepLabV3+.
- We did not train DeepLabV3 from scratch, so the effect of pretraining alone is not separated from the effect of the architecture.



3.6.4 Experimental and Reproducibility Limitations

- One seed per setting, so no mean ± std; gaps of 1–2 pp are not reliable.
- E0 and E2 compare runs trained on different GPUs and batch sizes, and in E1 full fine-tuning used a smaller backbone LR.
- No LR schedule and no hyperparameter search.
- cuDNN benchmark is on, so a re-run will not give bit-exact numbers.
- The E1 test results were made on Colab with their epoch-52/59/59 checkpoints. The `best.pt` files now in those folders come from an older 10-epoch run, so we run the error analysis and examples on the CE + Dice run, whose checkpoint we have locally.

3.6.5 Conclusion

Pretrained DeepLabV3-ResNet50 with partial fine-tuning raises test mIoU from 49.35% (U-Net) to 66.53% with CE and 68.09% with CE+Dice. It is 2.4 times slower but still real-time (about 60 FPS). The gain comes from big rare objects and region interiors. Border errors do not drop and are 63% of what is left, so better border handling and higher resolution are the next steps. Balanced losses help only a little.

Short answer to our question: yes, an ImageNet-pretrained DeepLabV3 is much better than a scratch U-Net on Cityscapes (+18.74 pp test mIoU), as long as the high backbone layers are fine-tuned. But pretraining does not fix borders and thin objects.

4 AI Usage Disclosure

All generative AI interactions for Assignment 2 are logged in `AI_USAGE.md` (Entries A2-0 to A2-15) in accordance with Course Handbook Section 5.4. Below is the concise list of AI assistance, tasks, and student verifications:

- **A2-0 to A2-2 — Architecture Scoping and Proposal (M1):** Gemini 3.8 Flash (Cao Duc Thanh) and ChatGPT / Claude (Dang Thanh Phu) were consulted for Cityscapes dataset exploration and understanding DeepLabV3-ResNet50. *Verification:* The team manually read the papers, checked Cityscapes licensing, and wrote the final proposal.

- **A2-3 to A2-7 — Pipeline, U-Net, Losses, and Bug Fixing (M2):** Gemini 3.8 Flash (Cao Duc Thanh) explained DataLoader syntax, provided code templates for U-Net and compound losses (`src/losses.py`), and suggested setting `drop_last=True` in `src/data.py` to fix a single-sample BatchNorm crash in ASPP. *Verification:* Code was tested locally, verified against PyTorch docs, and evaluated on an RTX 5070 Ti.

- **A2-8 — Cloud Training Notebook (M2/M3):** Claude Opus 5.5 (Dang Thanh Phu) assisted in merging standalone modules into a single Google Colab notebook (`colab.ipynb`) with Google Drive checkpoint resumption and automatic export. *Verification:* Ran end-to-end on Colab, confirmed training resumed properly from Drive, and validated output weights.

- **A2-9 & A2-10 — Evaluation and Demo Setup (M3):** Claude Opus 5.5 (Dang Thanh Phu) assisted in tabulating preliminary validation/test metrics from logs and structuring evaluation commands for the live model demo. *Verification:* Verified every table value against `log.csv` and `metrics_test.json`, and re-ran the evaluation on an RTX 4050.

- **A2-11 & A2-12 — Qualitative Plots and Error Analysis (M3):** Gemini 3.8 Flash (Cao Duc Thanh) provided logic for qualitative image selection (`src/visualize.py`), error mask generation, boundary error extraction, and confusion matrix plotting (`src/error_analysis.py`). *Verification:* Executed locally, inspected generated figures for label consistency, and verified matrix arithmetic.

- **A2-13 — Report Data Extraction and Benchmarking Scripts (M3):** Gemini 3.8 Flash (Cao Duc Thanh) assisted in implementing four automation scripts (`scripts/m3_training_summary.py`, `m3_error_qual.py`, `m3_eda_stats.py`, and `benchmark_cost.py`) to parse training logs, extract 4-class error taxonomies, compute Cityscapes dataset stats, and profile FLOPs and latency. *Verification:* Ran all four scripts on local GPUs, confirmed all generated JSON artifacts and figures matched raw data without synthetic fabrication, and copied the verified numbers into the report.

- **A2-14 & A2-15 — Report Structuring and Presentation Slides (M3):** Gemini 3.8 Flash (Cao Duc Thanh) assisted in organizing report sections to meet handbook guidelines, and Claude Opus 5.5 helped format presentation slides. *Verification:* The text was reviewed, rewritten to match experimental findings, and checked against rubric requirements.



Declaration of Academic Integrity: Every line of code was reviewed and executed by the team. All experimental metrics in this report originate from logged training runs and direct hardware measurements; no empirical figures were generated by AI. Both members take full intellectual ownership of this work.

References

[1] TorchVision, *ResNet-50*, TorchVision documentation [Online]. Available: https://docs.pytorch.org/vision/stable/models/generated/torchvision.models.resnet50.html.